Experiment 3: Build an Artificial Neural Network by implementing the Backpropagation algorithm and test thesame using appropriate data sets.

1.student plaement prediction dataset

CGPA	Aptitude Score	Communication Score	Placement
9.2	90	88	Placed
8.8	85	82	Placed
8.5	80	79	Placed
7.2	72	70	Not Placed
6.8	68	65	Not Placed
9	92	90	Placed
7.5	75	74	Not Placed
8.6	84	81	Placed
6.5	60	58	Not Placed
8.9	88	86	Placed

CODE:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler,LabelEncoder
from sklearn.metrics import accuracy_score,classification_report

data=pd.read_csv("student plaement prediction .csv")
X=data[["CGPA","Aptitude Score","Communication Score"]].values
y=data["Placement"].values
```

```
encoder=LabelEncoder()
y=encoder.fit_transform(y)

scaler=StandardScaler()
X=scaler.fit_transform(X)

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

input_size=X_train.shape[1]
hidden_size=5
np.random.seed(42)

W1=np.random.randn(input_size,hidden_size)*0.1
b1=np.zeros((1,hidden_size))
W2=np.random.randn(hidden_size,1)*0.1
b2=np.zeros((1,1))

lr=0.1
epochs=1000

def sigmoid(x):
    return 1/(1+np.exp(-np.clip(x,-500,500)))

def derivative(x):
    return x*(1-x)

for i in range(epochs):
    a1=sigmoid(X_train@W1+b1)
    a2=sigmoid(a1@W2+b2)
```

```
    error=y_train.reshape(-1,1)-a2
    d2=error*derivative(a2)
    d1=(d2@W2.T)*derivative(a1)
    W2+=lr*a1.T@d2
    b2+=lr*np.sum(d2,axis=0,keepdims=True)
    W1+=lr*X_train.T@d1
    b1+=lr*np.sum(d1,axis=0,keepdims=True)

a1=sigmoid(X_test@W1+b1)
a2=sigmoid(a1@W2+b2)
pred=(a2>=0.5).astype(int).flatten()

print("Actual:",y_test)
print("Predicted:",pred)
print("Accuracy:",accuracy_score(y_test,pred))
print(classification_report(y_test,pred,target_names=encoder.classes_))
```

OUTPUT:

```
Actual: [0 1]
Predicted: [0 1]
Accuracy: 1.0
              precision    recall  f1-score   support

  Not Placed       1.00      1.00      1.00         1
      Placed       1.00      1.00      1.00         1

    accuracy                           1.00         2
   macro avg       1.00      1.00      1.00         2
weighted avg       1.00      1.00      1.00         2
```

2. loan approval prediction dataset

Income (₹ Lakhs)	Credit Score	Loan Amount (₹ Lakhs)	Loan Approved
8	780	5	Yes
7	760	4	Yes
6	720	6	Yes
4	620	8	No
3	580	10	No
9	800	4	Yes
5	650	7	No
8	770	5	Yes
4	600	9	No
7	740	6	Yes

Code:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder,StandardScaler
from sklearn.metrics import accuracy_score,classification_report

data=pd.read_csv("loan approval prediction .csv")
data=data.dropna()

if "Loan_ID" in data.columns:
    data=data.drop(columns=["Loan_ID"])
```

```
X=data.drop(columns=["Loan Approved"]) # Corrected column name
y=data["Loan Approved"] # Corrected column name

for col in X.select_dtypes(include=["object"]).columns:
    X[col]=LabelEncoder().fit_transform(X[col])

encoder=LabelEncoder()
y=encoder.fit_transform(y)

X=X.astype(float).values
X=StandardScaler().fit_transform(X)

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

input_size=X_train.shape[1]
hidden_size=8
np.random.seed(42)

W1=np.random.randn(input_size,hidden_size)*0.1
b1=np.zeros((1,hidden_size))
W2=np.random.randn(hidden_size,1)*0.1
b2=np.zeros((1,1))

lr=0.05
epochs=1500

def sigmoid(x):
    return 1/(1+np.exp(-np.clip(x,-500,500)))
```

```
def derivative(x):
    return x*(1-x)


for i in range(epochs):
    a1=sigmoid(X_train@W1+b1)
    a2=sigmoid(a1@W2+b2)
    error=y_train.reshape(-1,1)-a2
    d2=error*derivative(a2)
    d1=(d2@W2.T)*derivative(a1)
    W2+=lr*a1.T@d2
    b2+=lr*np.sum(d2,axis=0,keepdims=True)
    W1+=lr*X_train.T@d1
    b1+=lr*np.sum(d1,axis=0,keepdims=True)


a1=sigmoid(X_test@W1+b1)
a2=sigmoid(a1@W2+b2)
pred=(a2>=0.5).astype(int).flatten()


print("Actual:",y_test)
print("Predicted:",pred)
print("Accuracy:",accuracy_score(y_test,pred))
print(classification_report(y_test,pred,target_names=encoder.classes_))
```

Output:

```
Actual: [0 1]
Predicted: [0 1]
Accuracy: 1.0
              precision    recall  f1-score   support

          No       1.00      1.00      1.00         1
         Yes       1.00      1.00      1.00         1

    accuracy                           1.00         2
   macro avg       1.00      1.00      1.00         2
weighted avg       1.00      1.00      1.00         2
```

3**. disease diagnosis**

Temperature (°C)	Heart Rate (bpm)	Oxygen Level (%)	Disease
39	110	94	Positive
38.5	108	95	Positive
37	78	99	Negative
39.2	115	93	Positive
36.8	75	98	Negative
38.8	112	94	Positive
37.2	80	98	Negative

Code:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder,StandardScaler
from sklearn.metrics import accuracy_score,classification_report

data=pd.read_csv("disease diagnosis .csv")
data=data.dropna()

X=data.drop(columns=["Disease"])
y=data["Disease"]

for col in X.select_dtypes(include=["object"]).columns:
    X[col]=LabelEncoder().fit_transform(X[col])

encoder=LabelEncoder()
```

```
y=encoder.fit_transform(y)

X=X.astype(float).values
X=StandardScaler().fit_transform(X)

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

input_size=X_train.shape[1]
hidden_size=10
np.random.seed(42)

W1=np.random.randn(input_size,hidden_size)*0.1
b1=np.zeros((1,hidden_size))
W2=np.random.randn(hidden_size,1)*0.1
b2=np.zeros((1,1))

lr=0.05
epochs=2000

def sigmoid(x):
    return 1/(1+np.exp(-np.clip(x,-500,500)))

def derivative(x):
    return x*(1-x)

for i in range(epochs):
    a1=sigmoid(X_train@W1+b1)
    a2=sigmoid(a1@W2+b2)
    error=y_train.reshape(-1,1)-a2
```

```
    d2=error*derivative(a2)
    d1=(d2@W2.T)*derivative(a1)
    W2+=lr*a1.T@d2
    b2+=lr*np.sum(d2,axis=0,keepdims=True)
    W1+=lr*X_train.T@d1
    b1+=lr*np.sum(d1,axis=0,keepdims=True)


a1=sigmoid(X_test@W1+b1)
a2=sigmoid(a1@W2+b2)
pred=(a2>=0.5).astype(int).flatten()


print("Actual:",y_test)
print("Predicted:",pred)
print("Accuracy:",accuracy_score(y_test,pred))
print(classification_report(y_test,pred,target_names=encoder.classes_,
labels=encoder.transform(encoder.classes_)))
```

Output:

```
Actual: [1 1]
Predicted: [1 1]
Accuracy: 1.0
              precision    recall  f1-score   support

    Negative       0.00      0.00      0.00         0
    Positive       1.00      1.00      1.00         2

    accuracy                           1.00         2
   macro avg       0.50      0.50      0.50         2
weighted avg       1.00      1.00      1.00         2
```

4. Employee Promotion Prediction dataset

Experience (Years)	Performance Score	Training Hours	Promotion
10	95	50	Yes
8	90	45	Yes
7	88	40	Yes
3	65	20	No
2	60	18	No
9	93	48	Yes
4	70	25	No
8	89	42	Yes
2	58	15	No
6	85	38	Yes

Code:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder,StandardScaler
from sklearn.metrics import accuracy_score,classification_report

data=pd.read_csv("Fruit Classification .csv")
data=data.dropna()

X=data.drop(columns=["Fruit"])
y=data["Fruit"]
```

```
for col in X.select_dtypes(include=["object"]).columns:
    X[col]=LabelEncoder().fit_transform(X[col])

encoder=LabelEncoder()
y=encoder.fit_transform(y)

X=X.astype(float).values
X=StandardScaler().fit_transform(X)

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

input_size=X_train.shape[1]
hidden_size=8
np.random.seed(42)

W1=np.random.randn(input_size,hidden_size)*0.1
b1=np.zeros((1,hidden_size))
W2=np.random.randn(hidden_size,1)*0.1
b2=np.zeros((1,1))

lr=0.05
epochs=1500

def sigmoid(x):
    return 1/(1+np.exp(-np.clip(x,-500,500)))

def derivative(x):
    return x*(1-x)
```

```
for i in range(epochs):
    a1=sigmoid(X_train@W1+b1)
    a2=sigmoid(a1@W2+b2)
    error=y_train.reshape(-1,1)-a2
    d2=error*derivative(a2)
    d1=(d2@W2.T)*derivative(a1)
    W2+=lr*a1.T@d2
    b2+=lr*np.sum(d2,axis=0,keepdims=True)
    W1+=lr*X_train.T@d1
    b1+=lr*np.sum(d1,axis=0,keepdims=True)


a1=sigmoid(X_test@W1+b1)
a2=sigmoid(a1@W2+b2)
pred=(a2>=0.5).astype(int).flatten()


print("Actual:",y_test)
print("Predicted:",pred)
print("Accuracy:",accuracy_score(y_test,pred))
print(classification_report(y_test,pred,target_names=encoder.classes_,
labels=encoder.transform(encoder.classes_)))
```

Output:

```
Actual: [0 0]
Predicted: [0 0]
Accuracy: 1.0
              precision    recall  f1-score   support

       Apple       1.00      1.00      1.00         2
      Orange       0.00      0.00      0.00         0

    accuracy                           1.00         2
   macro avg       0.50      0.50      0.50         2
weighted avg       1.00      1.00      1.00         2
```

5. Fruit Classification dataset

Weight (g)	Diameter (cm)	Sweetness (%)	Fruit
150	7.5	90	Apple
160	7.8	88	Apple
140	7.2	91	Apple
120	5.5	70	Orange
125	5.8	72	Orange
130	6	74	Orange
155	7.6	89	Apple
118	5.4	69	Orange
148	7.4	92	Apple
122	5.7	71	Orange

Code:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder,StandardScaler
from sklearn.metrics import accuracy_score,classification_report

data=pd.read_csv("Employee Promotion Prediction .csv")
data=data.dropna()

X=data.drop(columns=["Promotion"])
y=data["Promotion"]

for col in X.select_dtypes(include=["object"]).columns:
```

```
    X[col]=LabelEncoder().fit_transform(X[col])

encoder=LabelEncoder()
y=encoder.fit_transform(y)

X=X.astype(float).values
X=StandardScaler().fit_transform(X)

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

input_size=X_train.shape[1]
hidden_size=10
np.random.seed(42)

W1=np.random.randn(input_size,hidden_size)*0.1
b1=np.zeros((1,hidden_size))
W2=np.random.randn(hidden_size,1)*0.1
b2=np.zeros((1,1))

lr=0.05
epochs=1500

def sigmoid(x):
    return 1/(1+np.exp(-np.clip(x,-500,500)))

def derivative(x):
    return x*(1-x)

for i in range(epochs):
```

```
    a1=sigmoid(X_train@W1+b1)
    a2=sigmoid(a1@W2+b2)
    error=y_train.reshape(-1,1)-a2
    d2=error*derivative(a2)
    d1=(d2@W2.T)*derivative(a1)
    W2+=lr*a1.T@d2
    b2+=lr*np.sum(d2,axis=0,keepdims=True)
    W1+=lr*X_train.T@d1
    b1+=lr*np.sum(d1,axis=0,keepdims=True)

a1=sigmoid(X_test@W1+b1)
a2=sigmoid(a1@W2+b2)
pred=(a2>=0.5).astype(int).flatten()

print("Actual:",y_test)
print("Predicted:",pred)
print("Accuracy:",accuracy_score(y_test,pred))
print(classification_report(y_test,pred,target_names=encoder.classes_))
```

Output:

```
Actual: [0 1]
Predicted: [0 1]
Accuracy: 1.0
              precision    recall  f1-score   support

          No       1.00      1.00      1.00         1
         Yes       1.00      1.00      1.00         1

    accuracy                           1.00         2
   macro avg       1.00      1.00      1.00         2
weighted avg       1.00      1.00      1.00         2
```